

Elenkhos Collaboration Brief

What we are building

Elenkhos is one end-to-end AI systems capstone with two reviewable packages:

- e`lenkhos-train`: data curation, tokenizer, decoder tests, 70M lab pretraining, eval, Qwen3-1.7B SFT, and DPO.
- e`lenkhos-serve`: export, quantization, C++ serving adapter, scheduler, reliability tests, canary, rollback, and release qualification.

The product goal is a small reasoning model that asks useful first-principles questions before answering. The systems goal is a measured training-to-serving release pipeline.

Why the 70M lab model exists

E`lenkhos-70M-Lab` is not the product model. It is a controlled systems baseline:

- held-out web and philosophy perplexity floor;
- weak reasoning negative control against Qwen3-1.7B;
- MHA versus GQA and FineWeb-only versus domain-mix ablation target;
- CS336-style scaling-law prediction test;
- profiler, memory, checkpoint, throughput, and cost artifact;
- serving release fixture before the larger product model.

Do not market the 70M checkpoint as a competitive reasoning model.

Timeline

Execution window: June 22 to September 11, 2026.

Capacity: 20 hours per week, planned as 17 committed hours plus 3 reserve hours. Total committed work is 204 hours, with 36 hours held for slippage, interviews, debugging, and reruns.

Main sequence:

- Data contract and source acquisition.
- BPE tokenizer and local token shards.
- Model primitives and evaluation contract.
- Mac local training promotion.
- Runpod systems pilot and teacher pilot.
- Cloud A3 scaling and verified distillation data.
- Ablations and first Qwen3-1.7B SFT.
- Final 70M run and DPO.
- Export, quantization, and serving integration.
- Scheduler, admission control, and reliability.
- Cross-layer report and portfolio packaging.

- Freeze, demo, and application materials.

Parallel work is allowed only when artifacts are independent. GPU jobs are sequential by default; C++ service work, evaluation writing, data review, and reporting can proceed while GPU runs execute.

How to collaborate

Use the GitHub Pages website as the map and GitHub Issues as the work queue.

Every contribution should name:

- the lesson or stage it belongs to;
- the artifact it changes;
- the command or test that verifies it;
- the measured result or reason it is blocked.

Recommended issue labels:

- data
- tokenizer
- training
- eval
- posttraining
- serving
- scheduler
- reliability
- docs
- good-first-task
- blocked
- needs-review

Contribution rules

- Keep roadmap decisions in the website manifest and project docs traceable.
- Do not add new project scope without cutting equivalent work.
- Do not use paid cloud compute before the Mac promotion gate passes.
- Do not change the eval set after model tuning starts.
- Do not claim an improvement without a frozen baseline and a reproducible command.
- Prefer small PRs that close one lesson gate or produce one artifact.

Definition of done

The summer package is done when the site, repos, report, and demos show:

- deterministic data manifests and contamination checks;

- a frozen tokenizer and tested model ladder;
- 70M baseline numbers and ablations;
- Qwen3-1.7B SFT or DPO candidate selected or rejected by frozen metrics;
- GGUF export and Q8/Q4 comparison;
- C++ service parity, streaming, cancellation, and scheduler tests;
- one blocked release and one rollback drill;
- final cross-layer report with resume-ready measured claims.